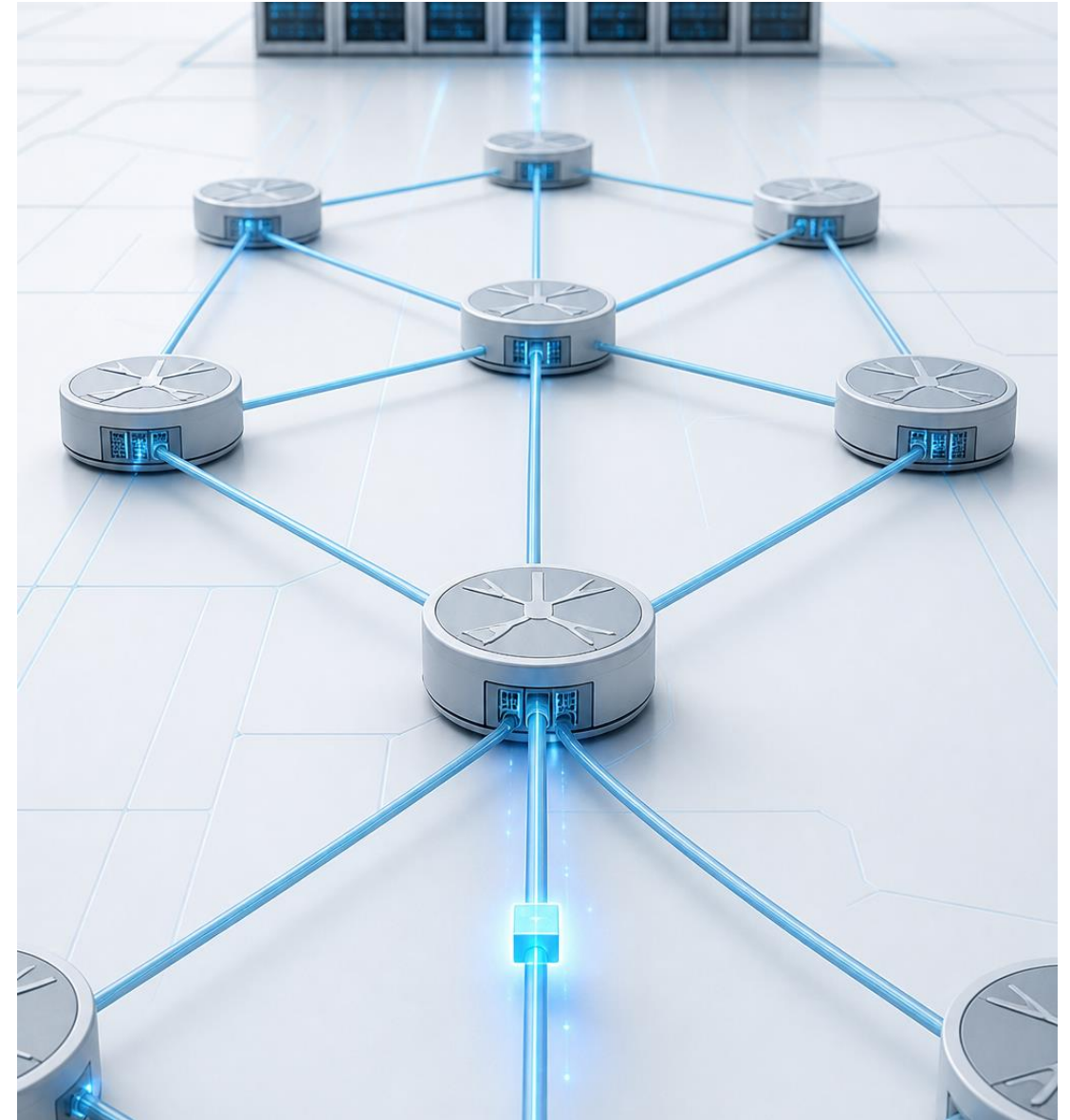


Chapter 1: Why the Internet Is Layered

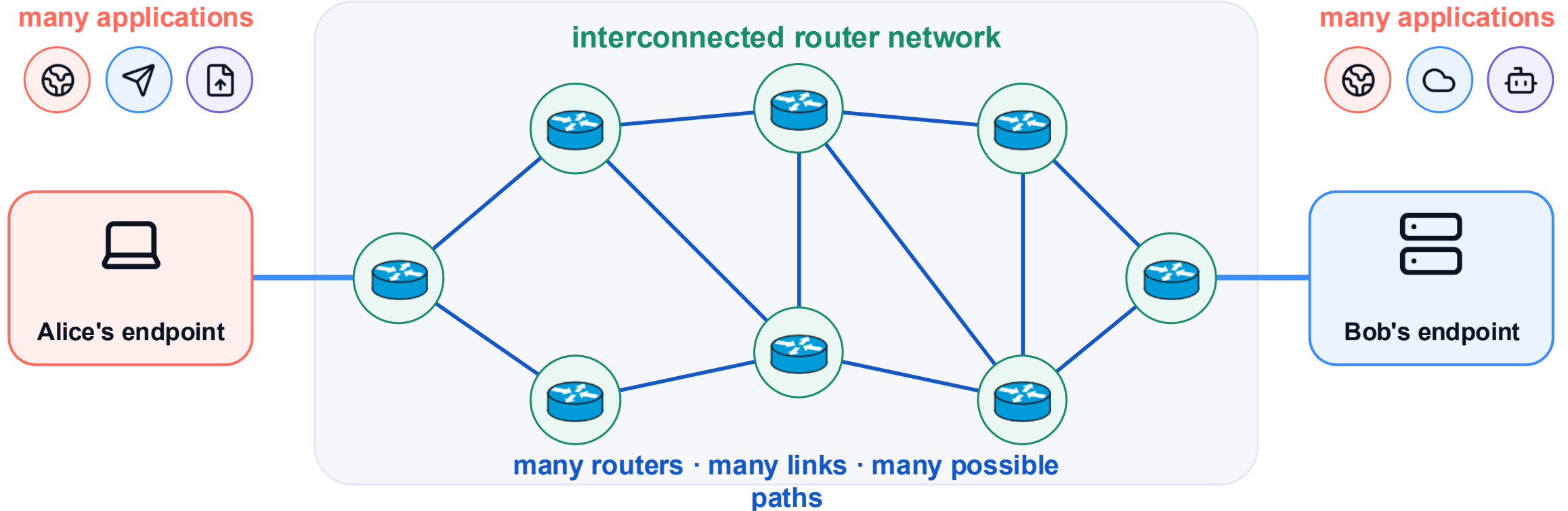
Abstraction, protocol stacks, and the end-to-end principle

Yaxiong Xie

CSE589 · Modern Network Concepts · Fall 2026



Last lecture gave us a scalable path between two endpoints



P2 showed how routers and paths scale end-to-end connectivity.

Need layers

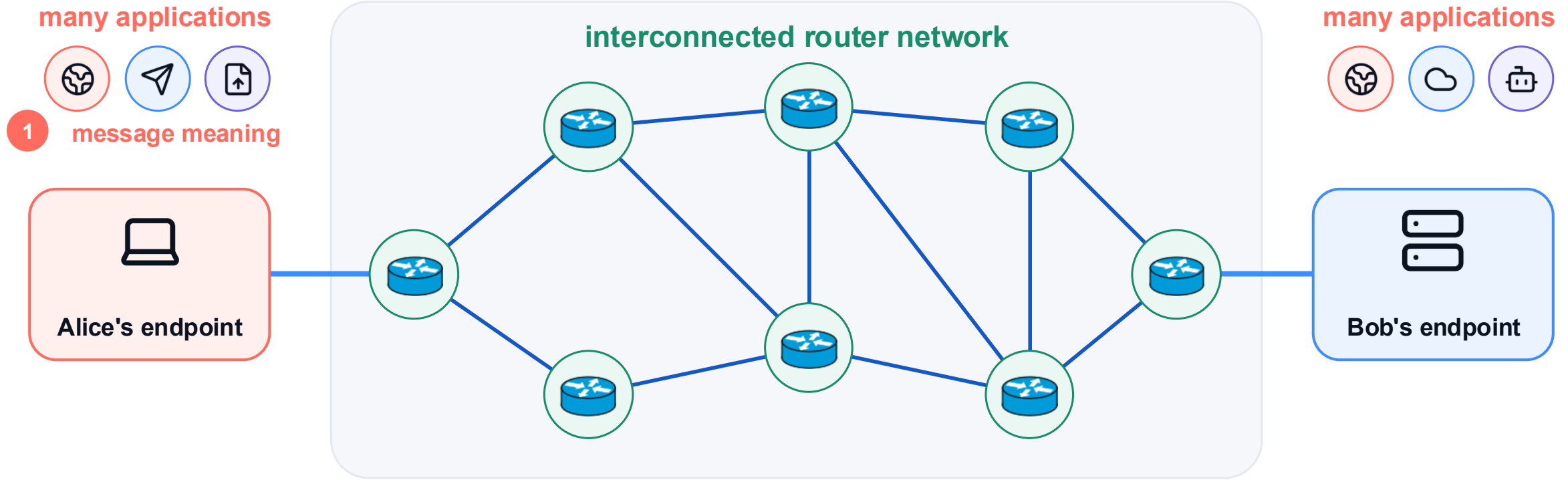
Define a layer

Implement layers

Place functions

Evolve

A network must handle many jobs for one delivery



1 Application logic decides what the message means.

Need layers

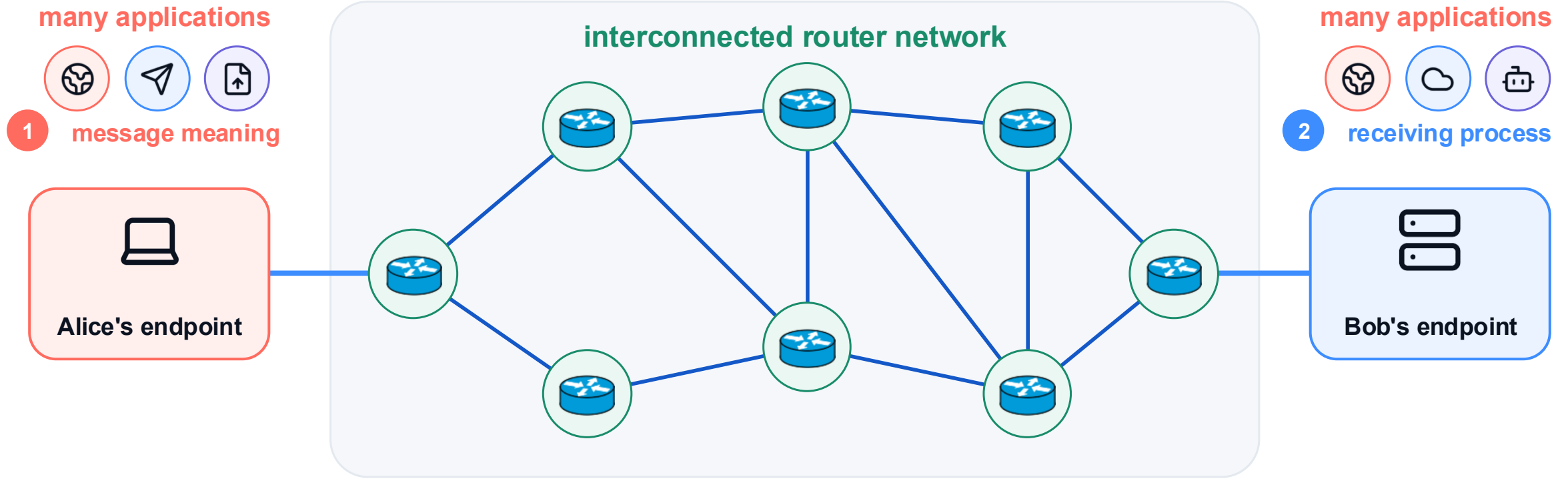
Define a layer

Implement layers

Place functions

Evolve

A network must handle many jobs for one delivery



2 Transport must deliver the message to the right receiving process.

Need layers

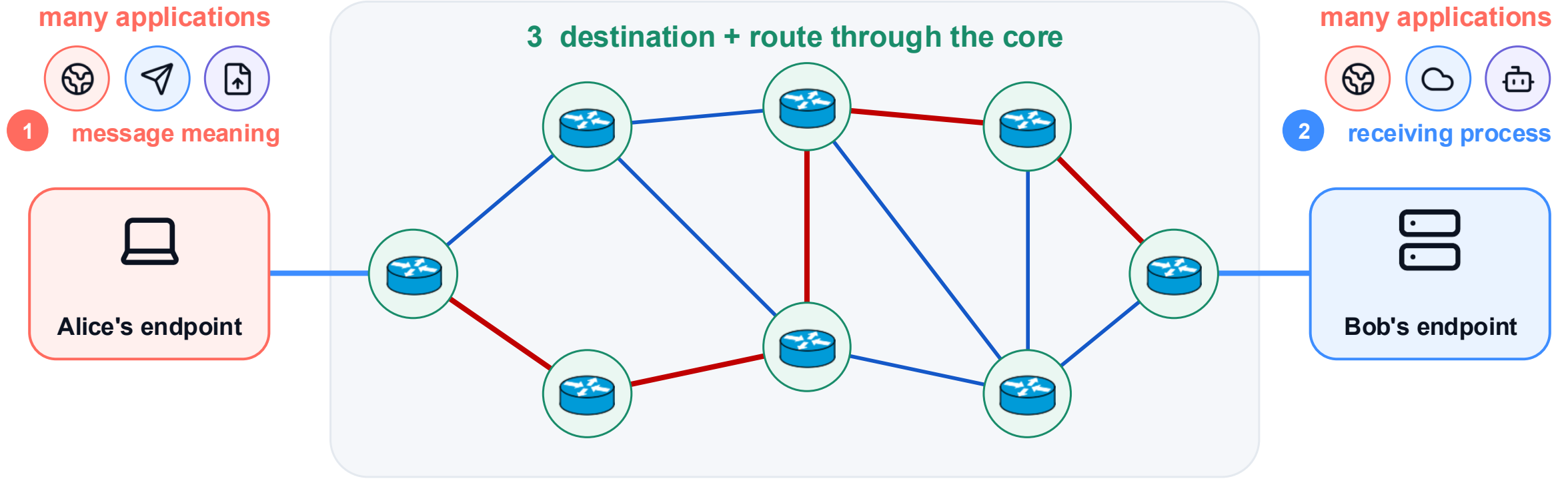
Define a layer

Implement layers

Place functions

Evolve

A network must handle many jobs for one delivery



3 Routing chooses the destination and a path through the core.

Need layers

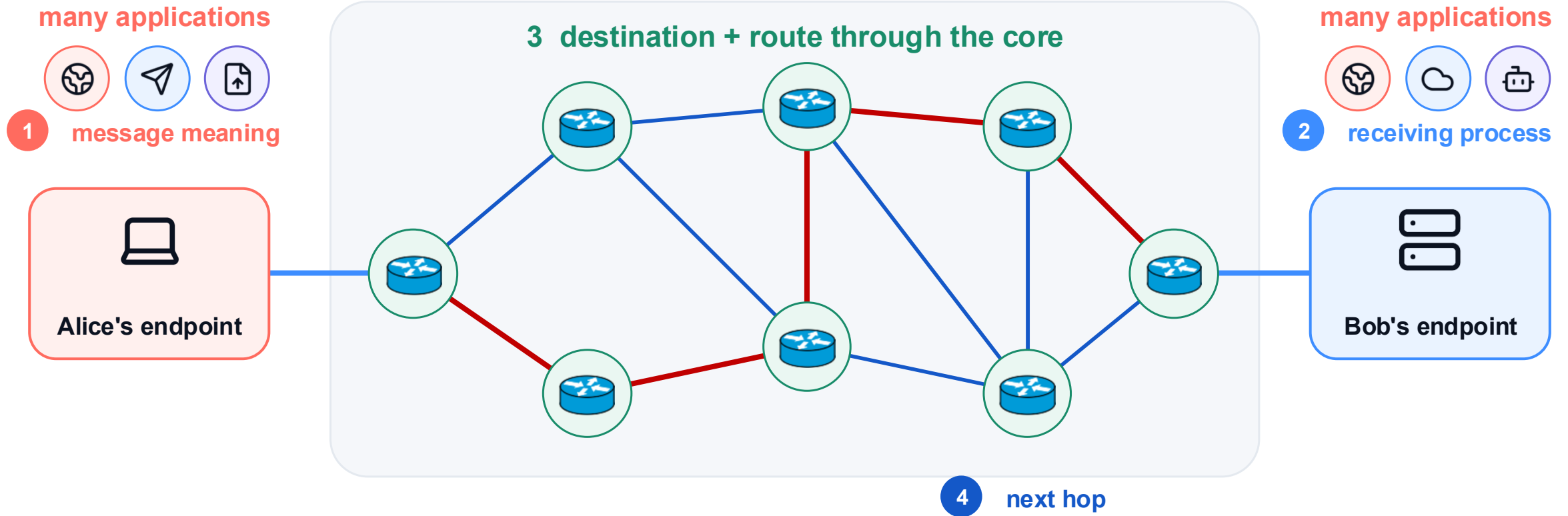
Define a layer

Implement layers

Place functions

Evolve

A network must handle many jobs for one delivery



4 Each router chooses one next hop for this packet.

Need layers

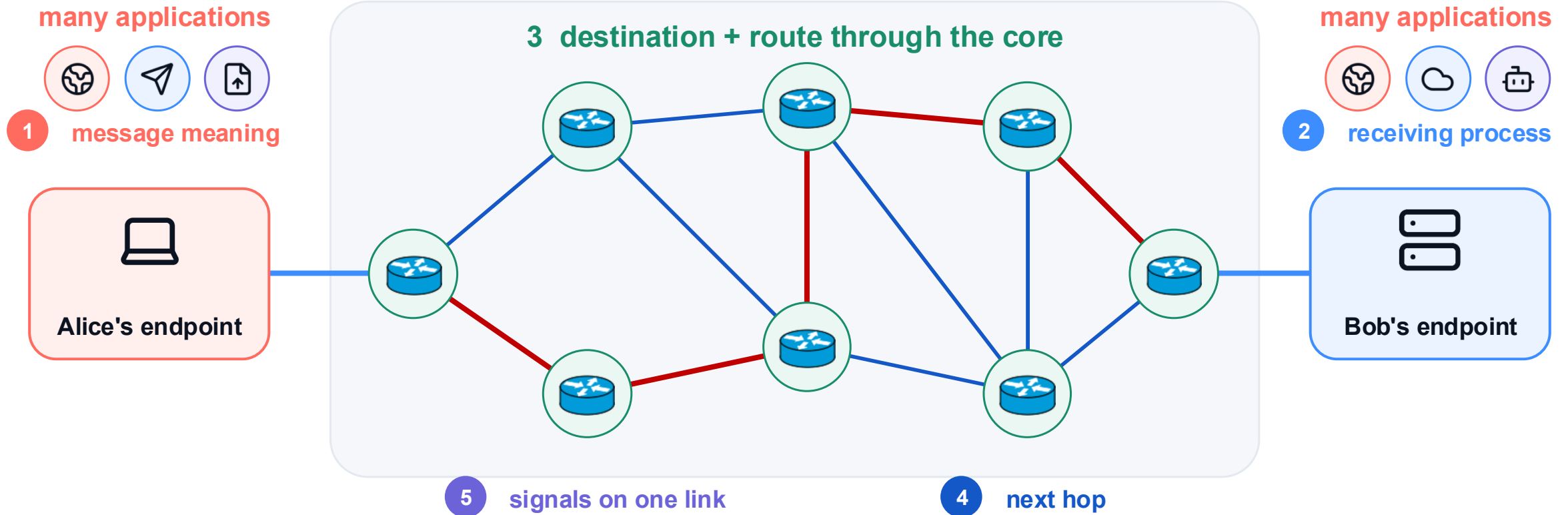
Define a layer

Implement layers

Place functions

Evolve

A network must handle many jobs for one delivery



5 Link and physical mechanisms carry signals across each hop.

Need layers

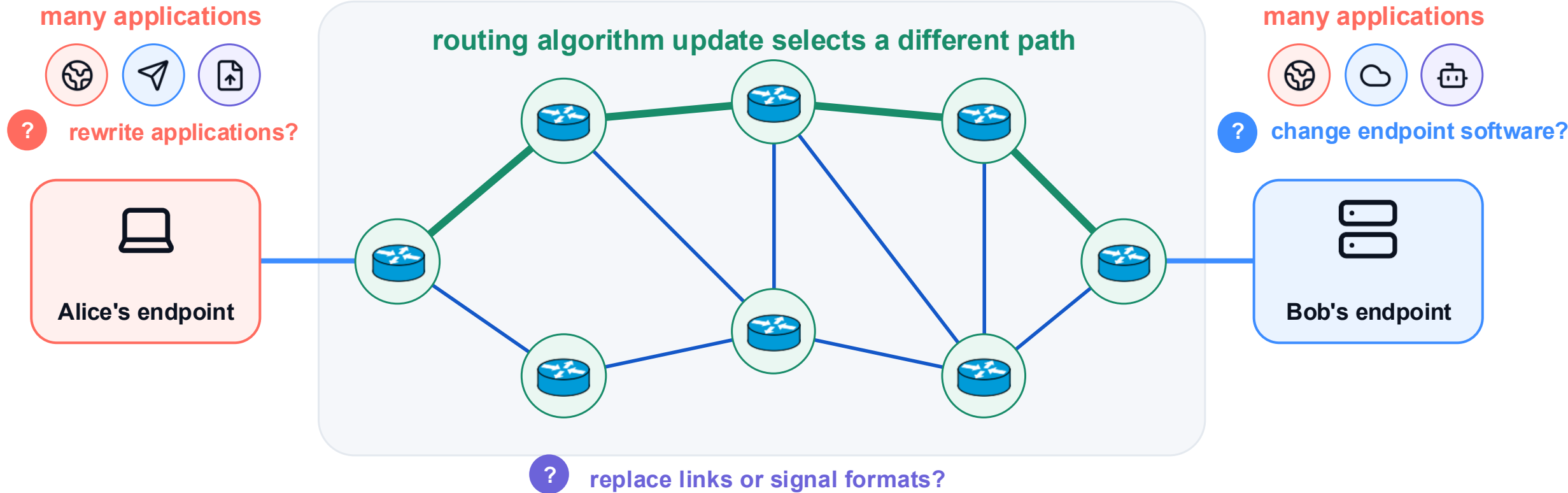
Define a layer

Implement layers

Place functions

Evolve

If routing changes, must applications and links change too?



Should one routing-policy change propagate through every component?

Need layers

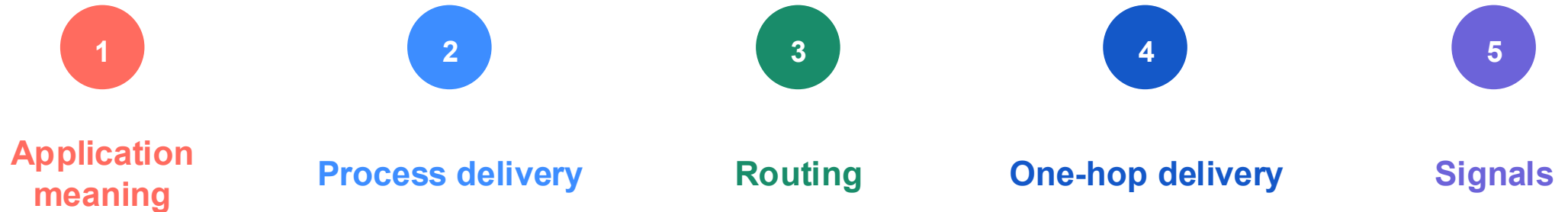
Define a layer

Implement layers

Place functions

Evolve

The question is where each responsibility should live



Who should own each responsibility—and what should everyone else be allowed to ignore?

Layering is an answer about responsibility and dependency—not just stacked boxes.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Part 1 · Why do we need layers?

01



**Separate responsibilities so
change stays local.**

Need layers

Define a layer

Implement layers

Place functions

Evolve

Start with one goal: deliver data to Bob's application

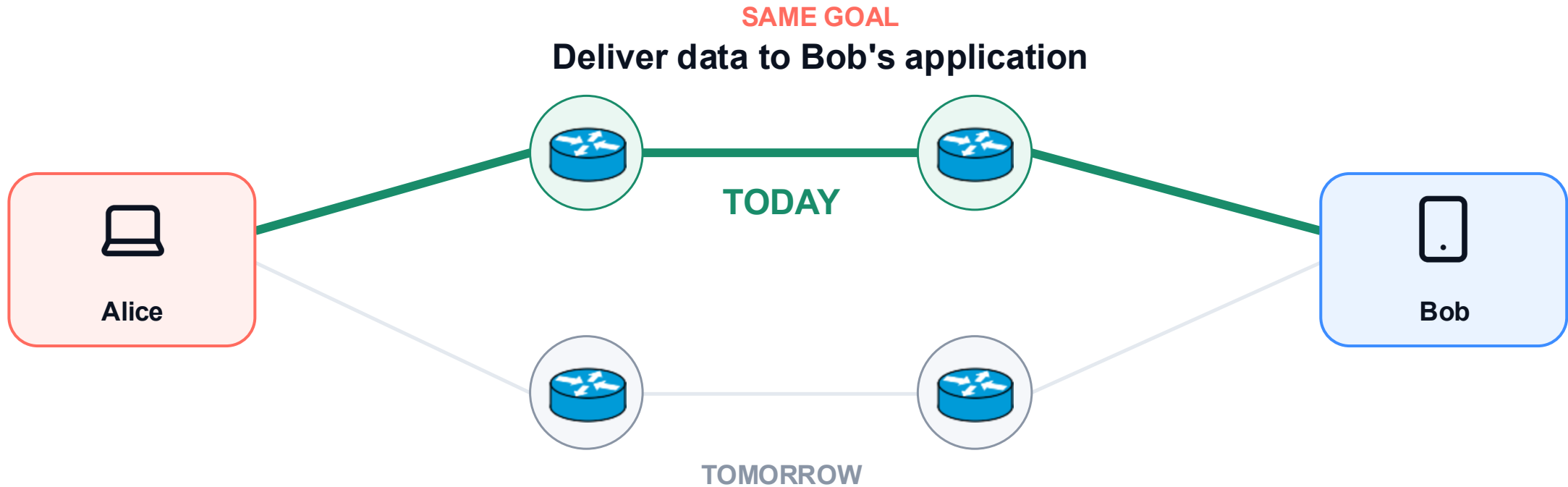


Application expectation: deliver this data to Bob's application.

The application wants a useful result—not a map of the network.



The path may change while the application goal stays fixed



Today, the network uses the upper route to reach Bob.

Need layers

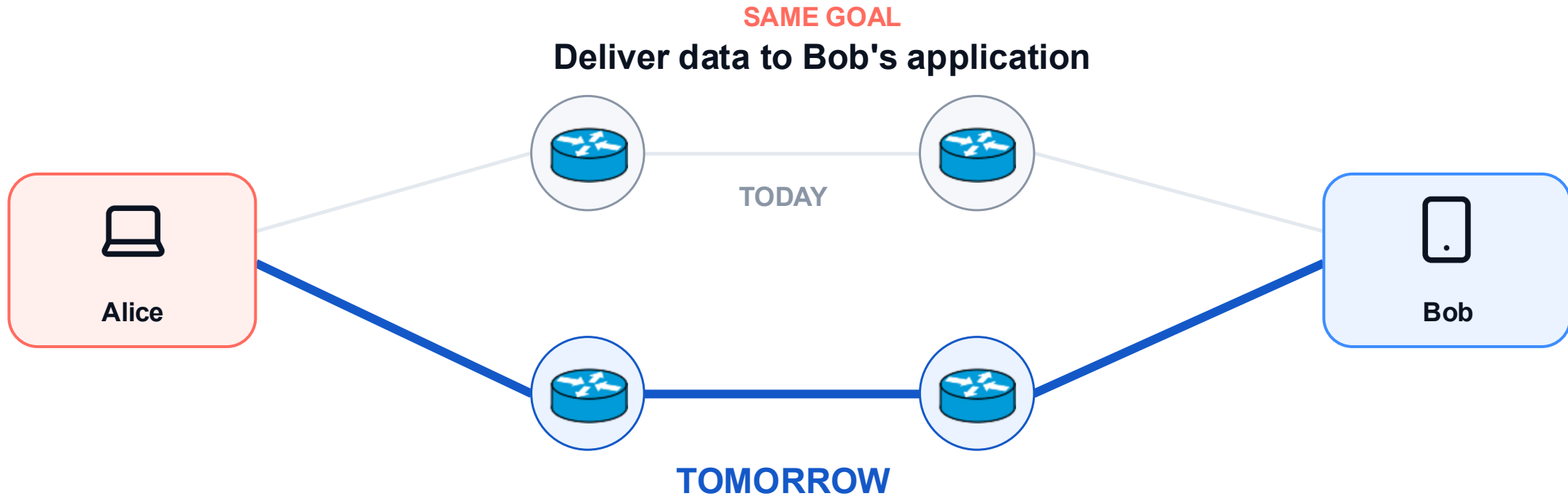
Define a layer

Implement layers

Place functions

Evolve

The path may change while the application goal stays fixed



Tomorrow, routing may choose another path—the application goal does not change.

Need layers

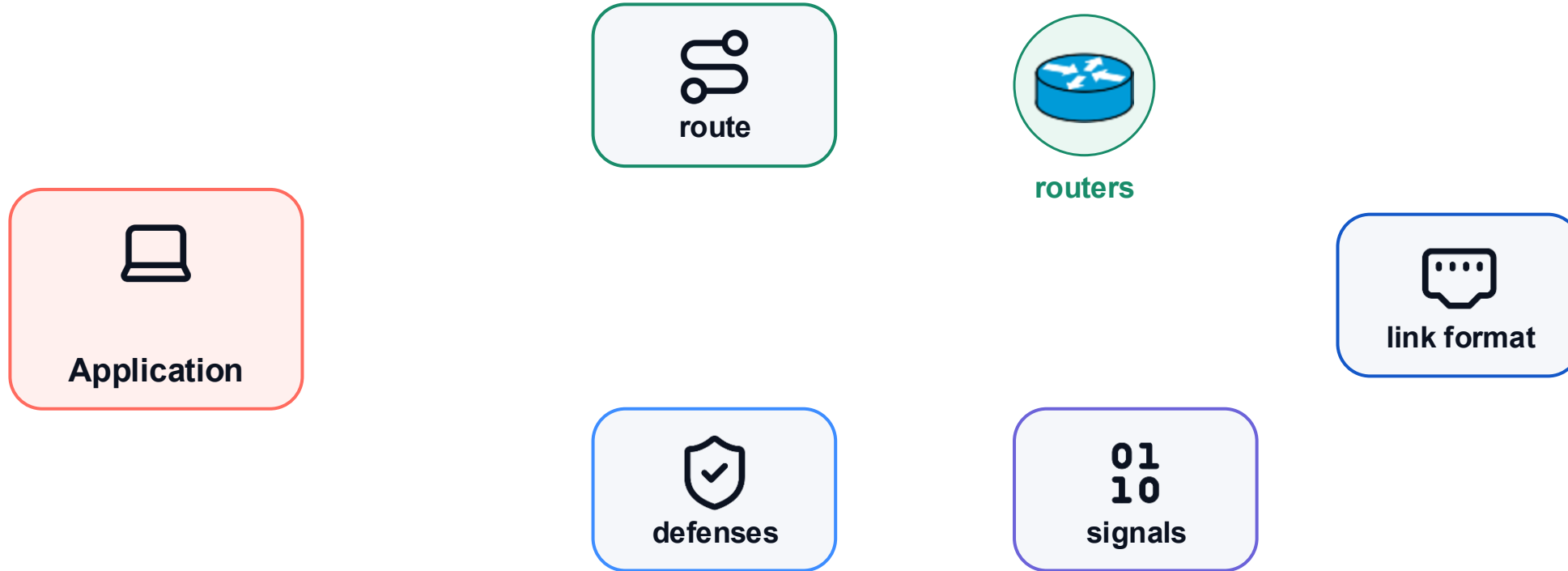
Define a layer

Implement layers

Place functions

Evolve

A monolith exposes network details to every application



Need layers

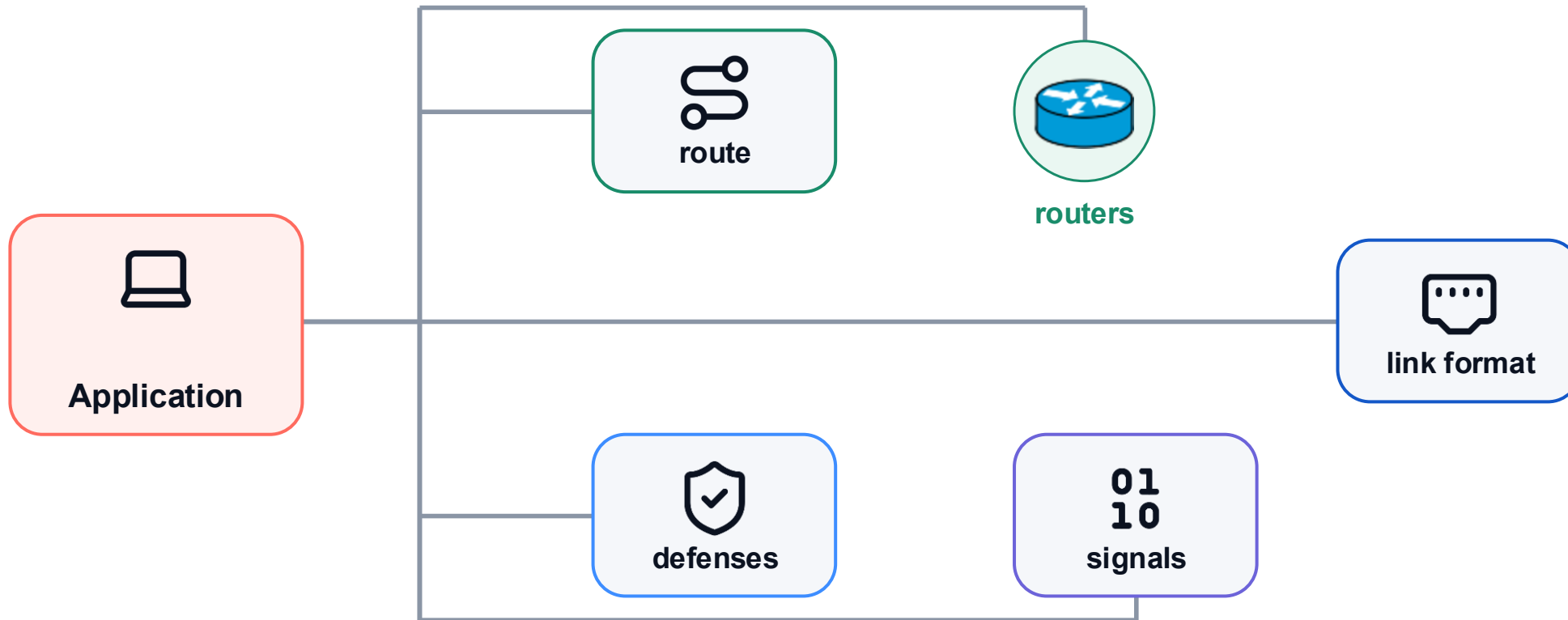
Define a layer

Implement layers

Place functions

Evolve

A monolith exposes network details to every application



Need layers

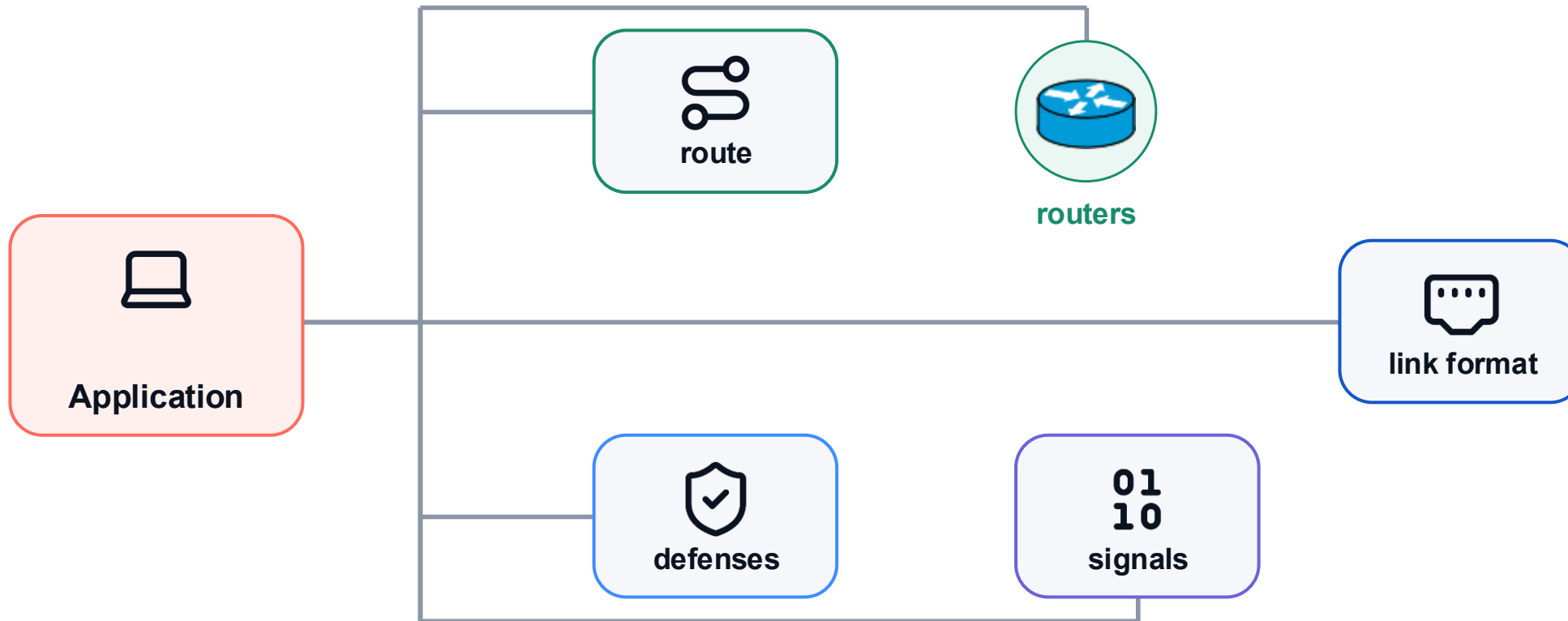
Define a layer

Implement layers

Place functions

Evolve

A monolith exposes network details to every application



Must the application understand every network detail?
If one protocol owns every detail, every change can force an application rewrite.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Changing Wi-Fi forces a monolithic application to change

TODAY · WI-FI ACCESS



```
app_send(data, Bob, Wi-Fi, route, recovery)
```

The application code names Wi-Fi—even though Wi-Fi is only the access technology.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Changing Wi-Fi forces a monolithic application to change

TODAY · WI-FI ACCESS



```
app_send(data, Bob, Wi-Fi, route, recovery)
```



AFTER MOVE · FIBER-OPTIC ACCESS



```
app_send(data, Bob, fiber, route, recovery)
```

Wi-Fi → fiber-optic access: the application must be rewritten although its goal is unchanged.

Need layers

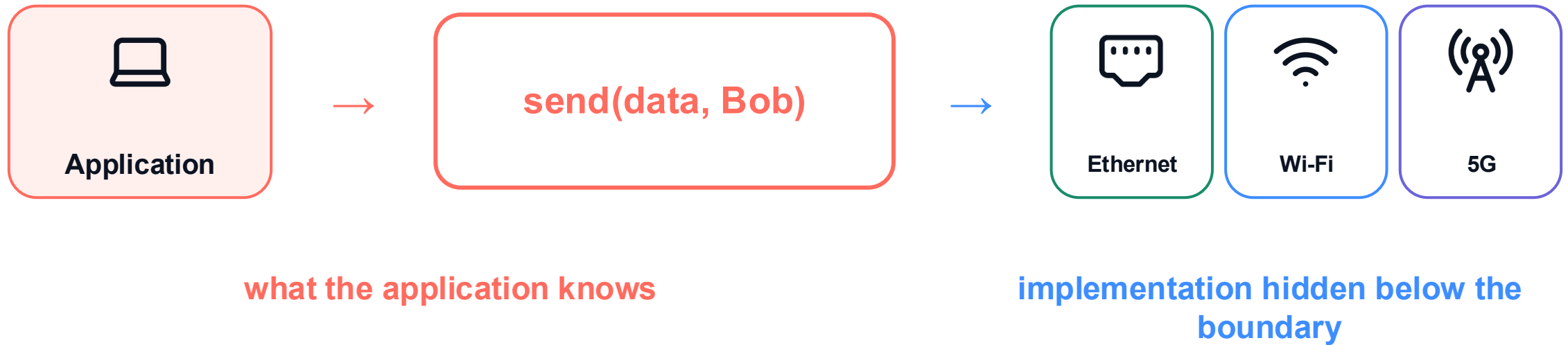
Define a layer

Implement layers

Place functions

Evolve

A good boundary hides the implementation below it



A useful abstraction reveals the result and hides the mechanism.

Need layers

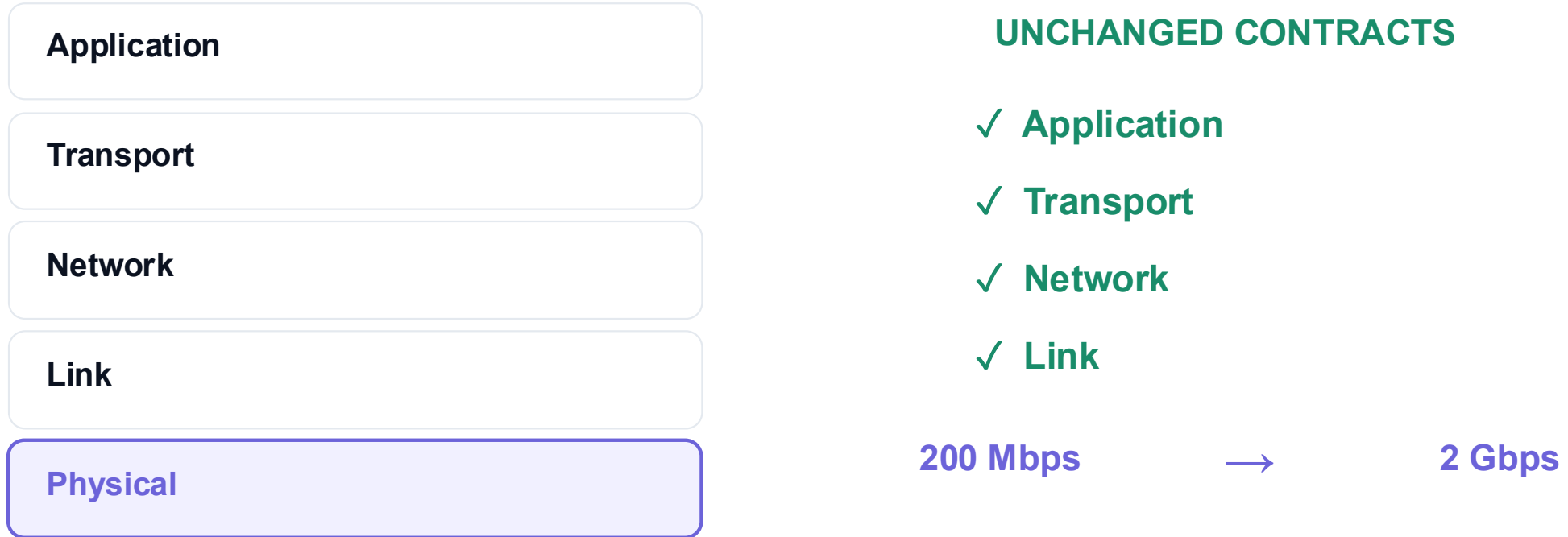
Define a layer

Implement layers

Place functions

Evolve

Stable boundaries let implementations change alone



A better physical implementation can preserve every upper-layer interface.



Each layer groups several related functions

FIVE LAYERS · ONE FOCUS AT A TIME

Application

Transport

Network

Link

Physical



CURRENT LAYER · 1 / 5

Application

Defines meaning and behavior for users

- ✓ Define message semantics
- ✓ Interpret user intent
- ✓ Choose application behavior

Application gives exchanged data meaning and chooses user-visible behavior.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Each layer groups several related functions

FIVE LAYERS · ONE FOCUS AT A TIME

✓ Application

Transport

Network

Link

Physical

CURRENT LAYER · 2 / 5



Transport

Delivers data between endpoint processes

- ✓ Identify the destination process
- ✓ Multiplex concurrent application flows
- ✓ Provide reliability and ordering

Transport identifies processes and controls their delivery behavior.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Each layer groups several related functions

FIVE LAYERS · ONE FOCUS AT A TIME

✓ Application

✓ Transport

Network

Link

Physical



CURRENT LAYER · 3 / 5

Network

Moves packets across interconnected networks

- ✓ Assign logical destination addresses
- ✓ Select end-to-end routes
- ✓ Forward packets hop by hop

Network addresses destinations, chooses routes, and forwards each packet.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Each layer groups several related functions

FIVE LAYERS · ONE FOCUS AT A TIME

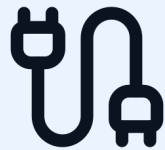
✓ Application

✓ Transport

✓ Network

Link

Physical



CURRENT LAYER · 4 / 5

Link

Moves one frame across one local link

- ✓ Frame packets for the local link
- ✓ Address the next-hop neighbor
- ✓ Detect local transmission errors

Link prepares and checks a frame for one neighboring hop.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Each layer groups several related functions

FIVE LAYERS · ONE FOCUS AT A TIME

✓ Application

✓ Transport

✓ Network

✓ Link

Physical



CURRENT LAYER · 5 / 5

Physical

Represents bits as signals on a medium

- ✓ Encode bits as physical signals
- ✓ Synchronize timing on the medium
- ✓ Transmit over copper, fiber, or radio

Physical converts bits into timed signals on the transmission medium.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Part 2 · What exactly is a layer?

02



What is promised?
How is it requested?
How do peers coordinate?

Need layers

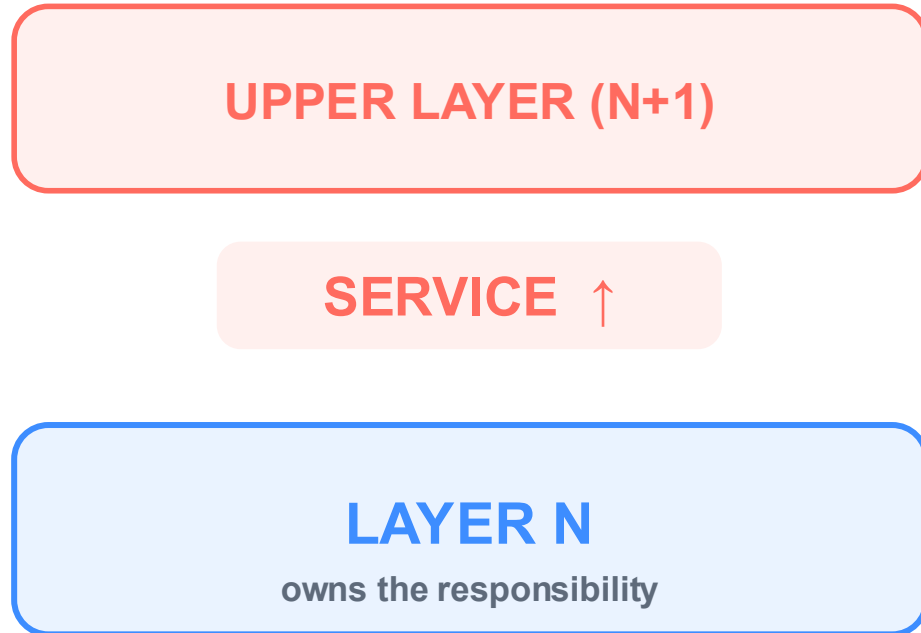
Define a layer

Implement layers

Place functions

Evolve

A service is the promise to the layer above



DEFINITION

A service is the capability or observable behavior a layer offers upward.

WHAT IT DOES

Tells the upper layer what it may rely on—without exposing how it is produced.

ONE EXAMPLE

“Deliver packets to the destination host.”

Service answers: WHAT does this layer provide?

Need layers

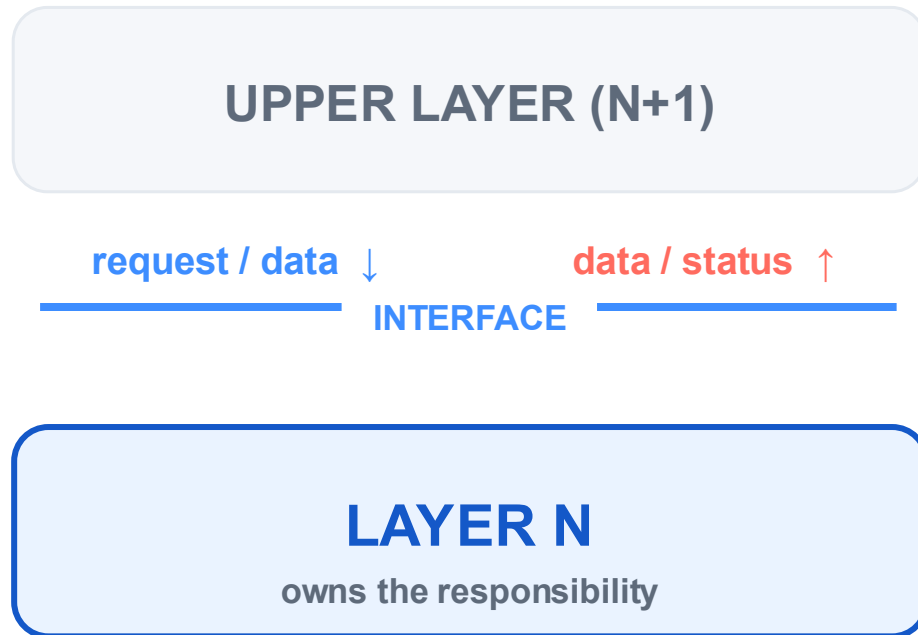
Define a layer

Implement layers

Place functions

Evolve

An interface is the local way to use the service



DEFINITION

An interface is the local boundary and operations used to access a service.

WHAT IT DOES

Specifies local requests, parameters, returned data, and status between adjacent layers.

ONE EXAMPLE

`send(data, destination)` ↓ ↑ `data / status`

Interface answers: HOW do adjacent layers interact here?

Need layers

Define a layer

Implement layers

Place functions

Evolve

A protocol is the shared rulebook between peers



A protocol is the shared message format and behavioral rules used by peer entities.

It specifies messages, their order, and the actions each peer must take.

Protocol answers: HOW do peers coordinate across systems?

Need layers

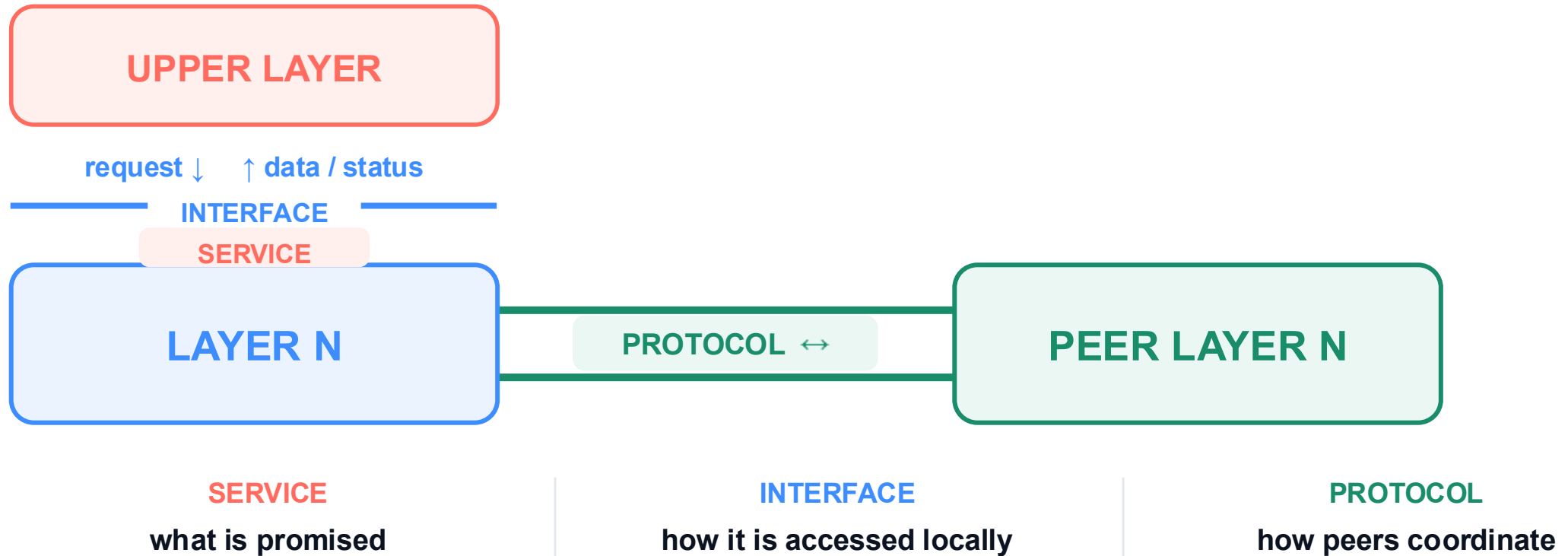
Define a layer

Implement layers

Place functions

Evolve

One layer, three different relationships



The layer is the module; the other three describe its relationships.

Need layers

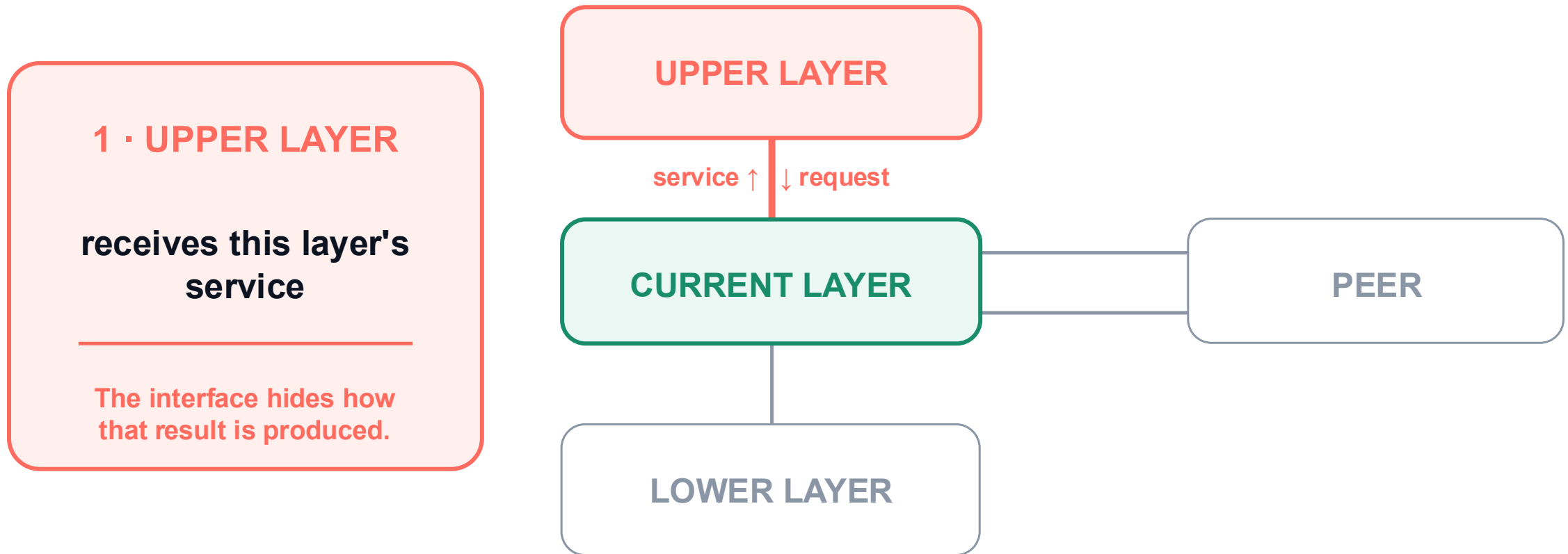
Define a layer

Implement layers

Place functions

Evolve

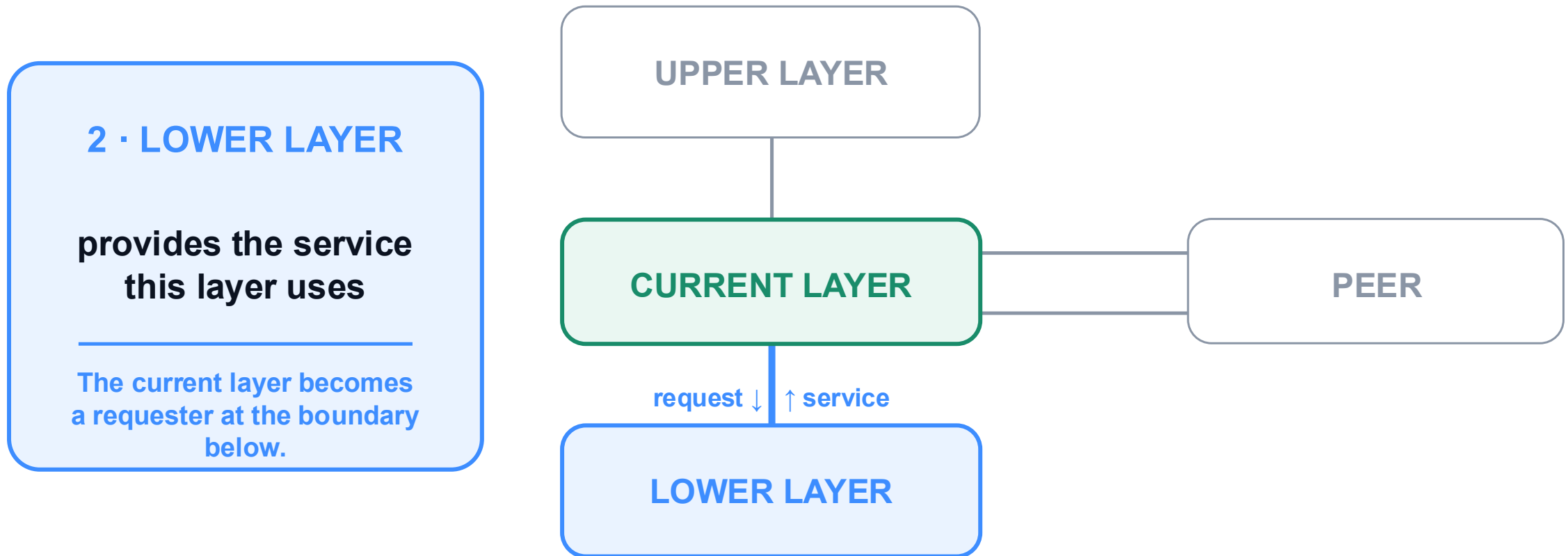
A stack is an ordered chain of layer contracts



Upward: provide a service behind a stable interface.



A stack is an ordered chain of layer contracts



Downward: request a simpler service from the layer below.

Need layers

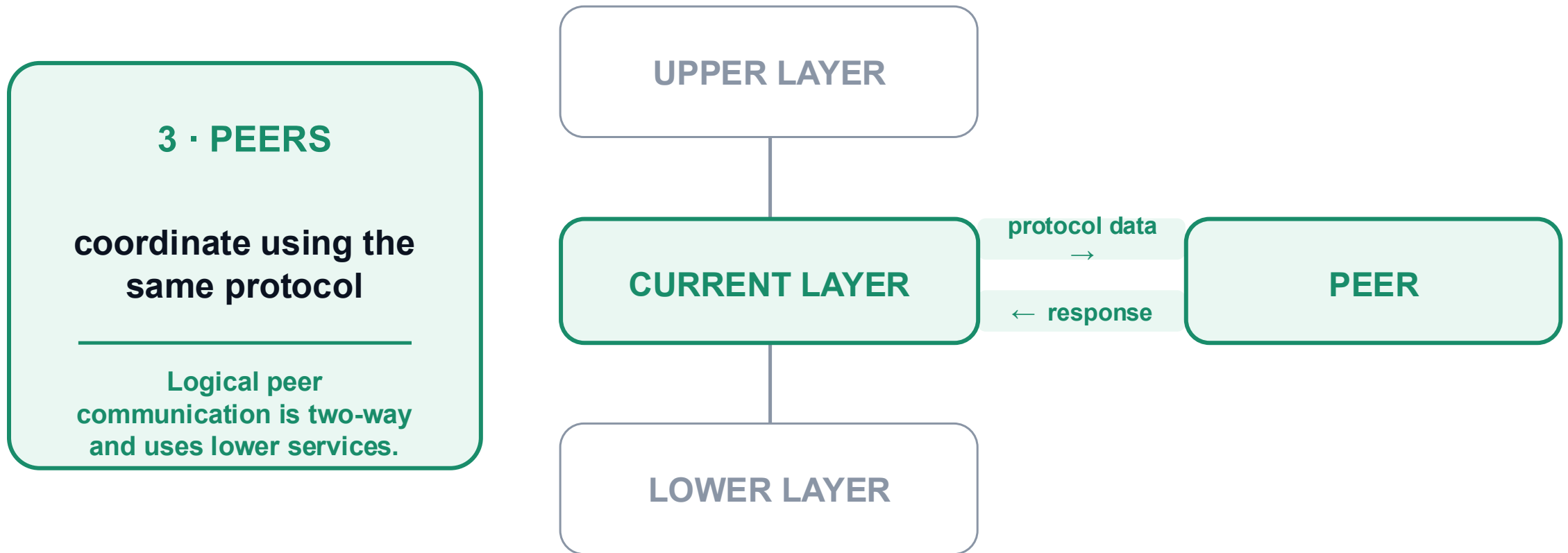
Define a layer

Implement layers

Place functions

Evolve

A stack is an ordered chain of layer contracts



Sideways: peers coordinate logically with a two-way protocol.

Need layers

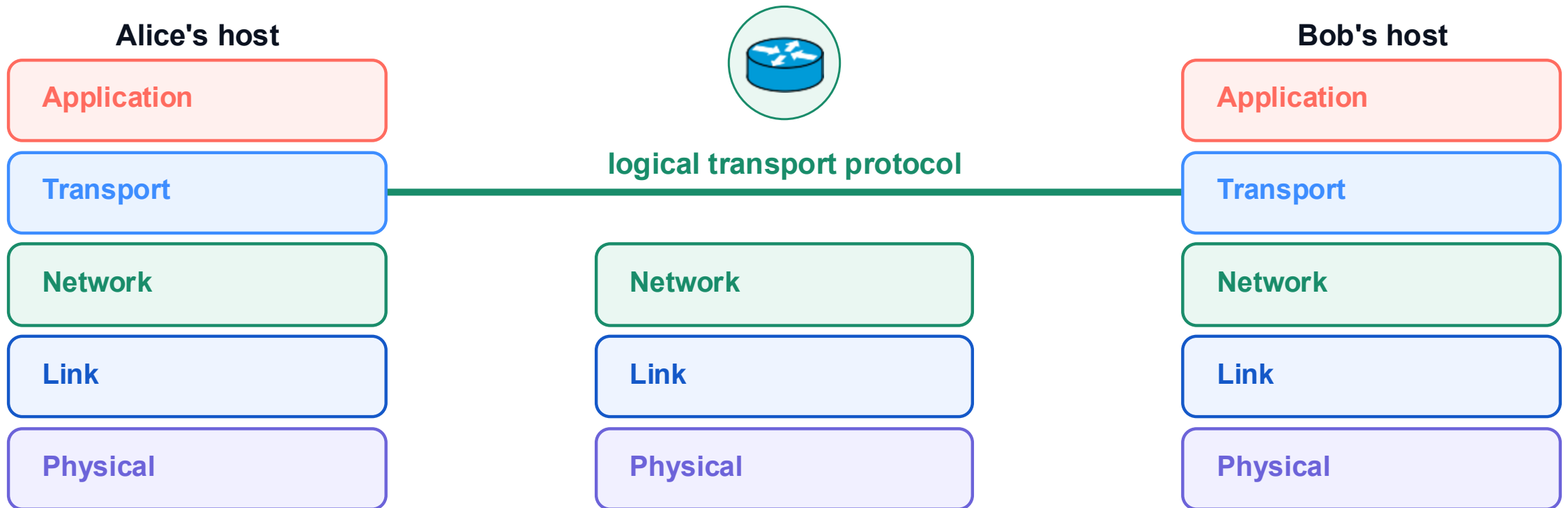
Define a layer

Implement layers

Place functions

Evolve

Peers communicate logically across the same layer



Peer protocols are a logical view: each side behaves as if it talks to its counterpart.

Need layers

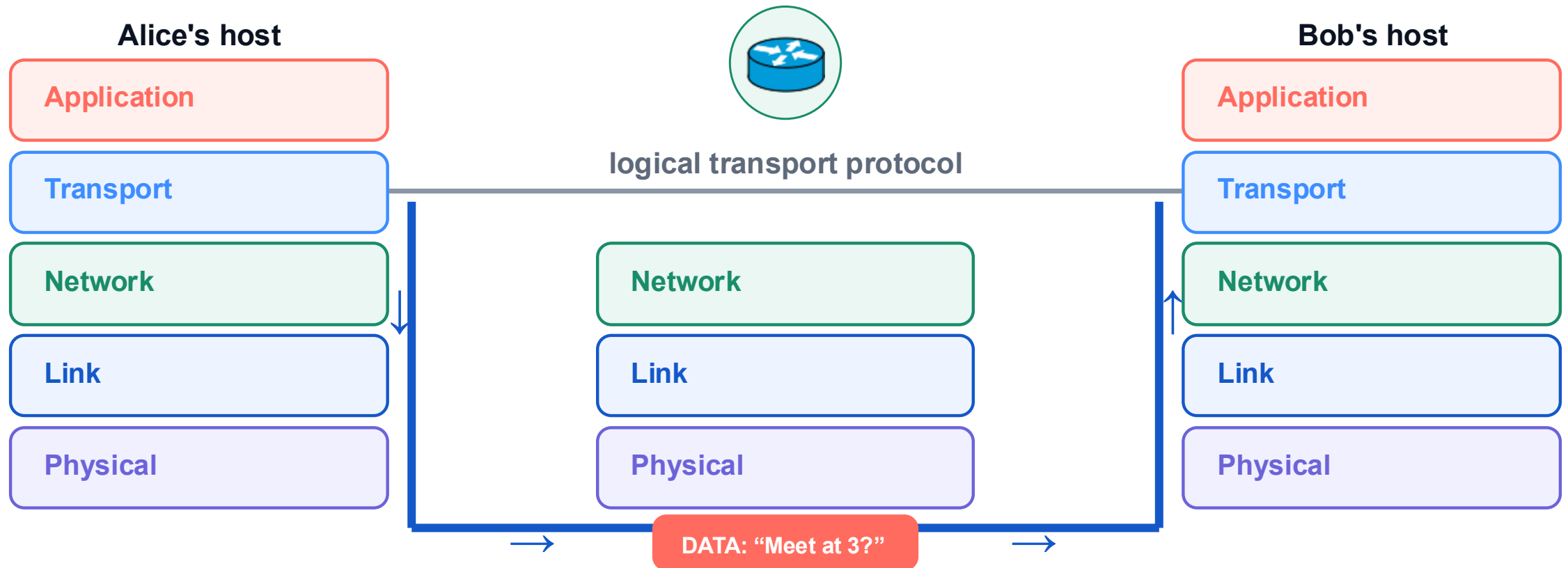
Define a layer

Implement layers

Place functions

Evolve

Real data travels down, across, and back up



The logical conversation is implemented by services from every lower layer.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Layering reduces coupling—but every boundary has a cost



WHAT LAYERING BUYS

Lower coupling across modules

✓ **Simpler reasoning**

Build, test, and debug one responsibility at a time.

✓ **Independent evolution**

Replace one layer without redesigning the rest—if its boundary stays stable.

✓ **Interoperability**

Common interfaces and protocols let independently built systems communicate.

Use layering when stable boundaries save more design effort than they add runtime cost.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Layering reduces coupling—but every boundary has a cost



WHAT LAYERING BUYS

Lower coupling across modules

✓ **Simpler reasoning**

Build, test, and debug one responsibility at a time.

✓ **Independent evolution**

Replace one layer without redesigning the rest—if its boundary stays stable.

✓ **Interoperability**

Common interfaces and protocols let independently built systems communicate.



WHAT LAYERING COSTS

Runtime and design-time penalties

• **Runtime overhead**

Every boundary may add headers, copies, checks, and processing.

• **Lost context**

A lower layer may miss application information that could improve performance.

• **Rigid abstractions**

A poor boundary creates workarounds and becomes expensive to change later.

Use layering when stable boundaries save more design effort than they add runtime cost.

Need layers

Define a layer

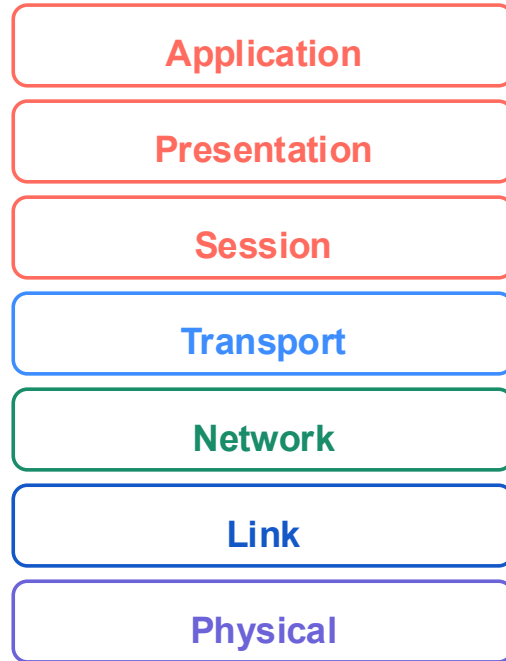
Implement layers

Place functions

Evolve

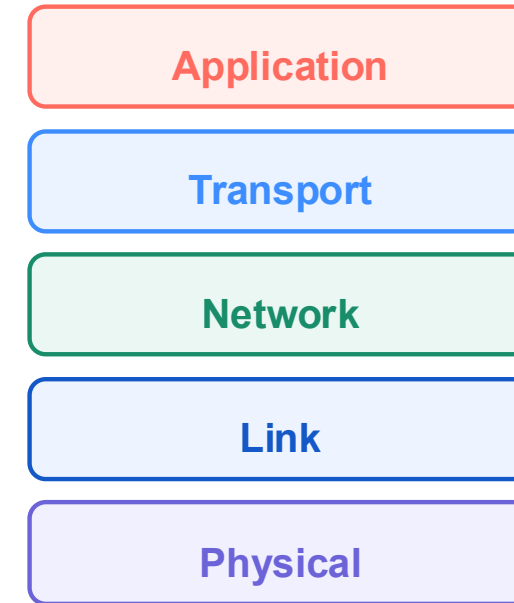
Five layers are a design choice, not a law of nature

OSI REFERENCE MODEL



session + presentation
handled inside applications
when needed

INTERNET STACK



Layer count follows design choices about boundaries; it is not a law of nature.

Need layers

Define a layer

Implement layers

Place functions

Evolve

Good boundaries minimize dependencies between modules

COHERENT

Related functions
behind one boundary



STABLE

A clear interface to the
layer above



REPLACEABLE

Implementation can
change behind the
contract

Layering manages complexity by containing dependencies—not by removing work.

Need layers

Define a layer

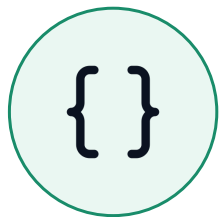
Implement layers

Place functions

Evolve

Part 3 · How is a layer implemented?

03



**Turn layer contracts into
running machinery.**

A stack diagram defines contracts—not running machinery

THE ARCHITECTURE

- Application
- Transport
- Network
- Link
- Physical



RUNNING MACHINERY

CODE RUNS

instructions execute

STATE CHANGES

tables and buffers evolve

DATA MOVES

units enter and leave

The stack names the contracts; implementation makes those contracts run.

A layer implementation combines four pieces

One layer needs four things to do real work



INTERFACE CALL

how work enters



+

PROCESSING CODE

what the layer does

+



LOCAL STATE

what this device
remembers

+

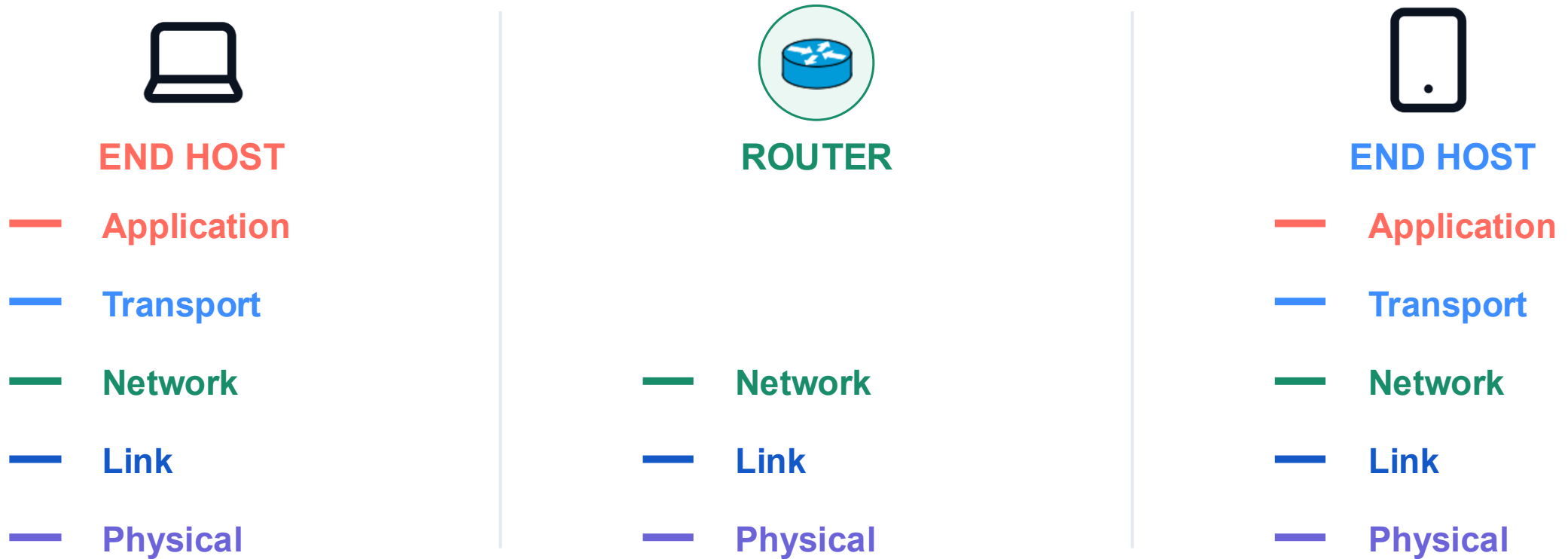


PEER PROTOCOL

how peers coordinate

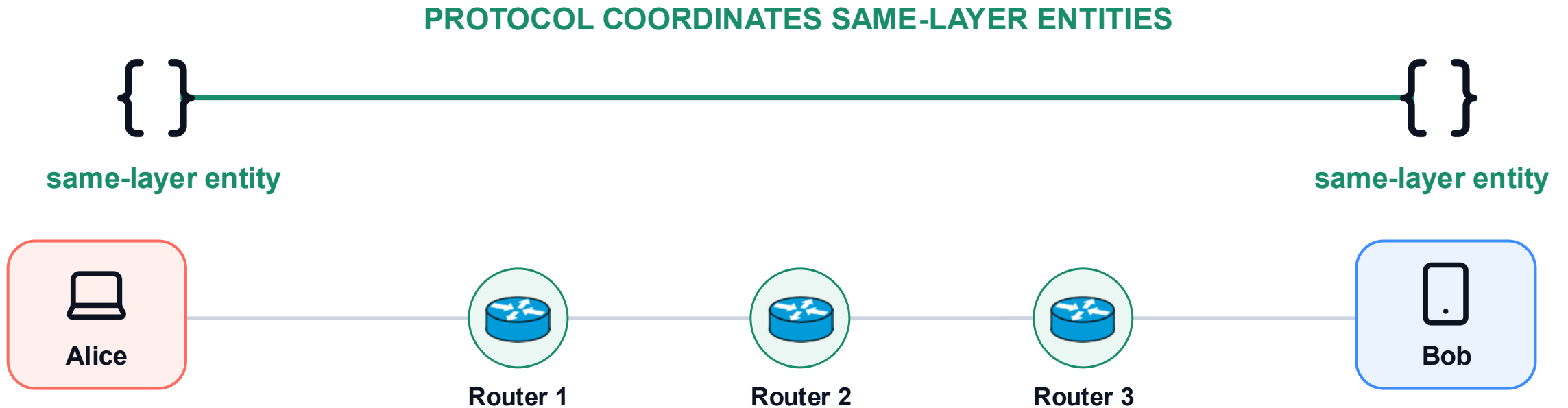
A protocol is one part of an implementation—not the whole implementation.

Different layers run in different places



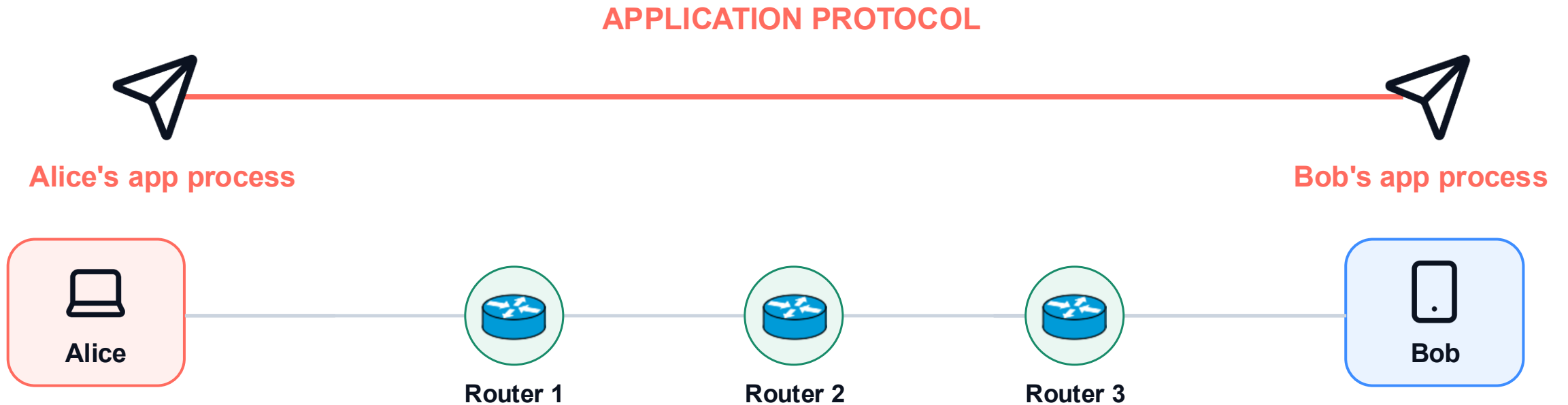
A layer is an abstraction; its code may run in software, firmware, or hardware.

A protocol coordinates peer entities at the same layer



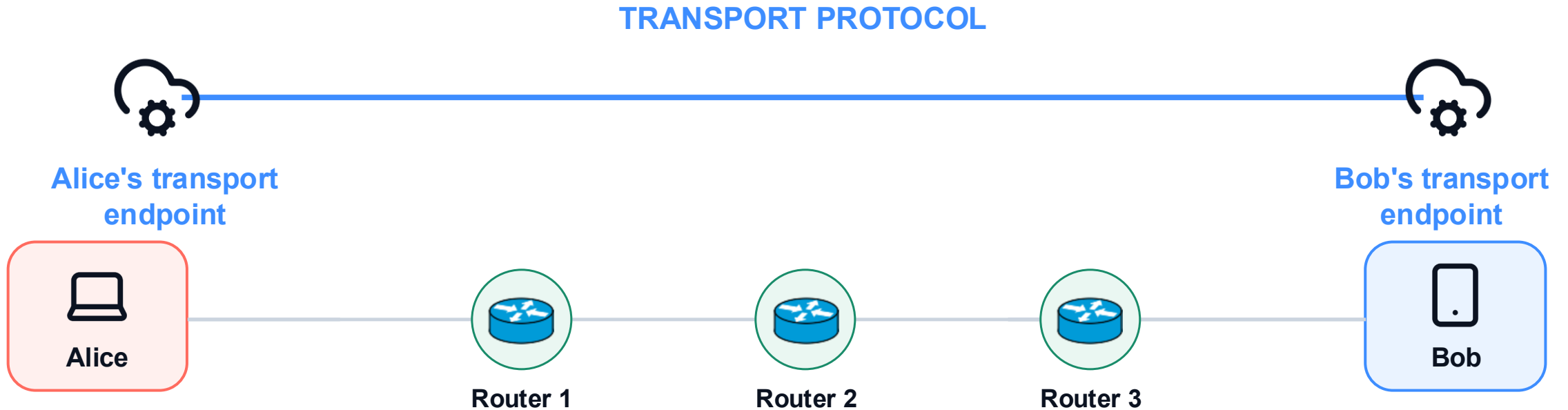
A protocol describes communication between peer entities; lower layers carry the real data.

Application peers are application processes



The application peer is the remote application process—not every device on the path.

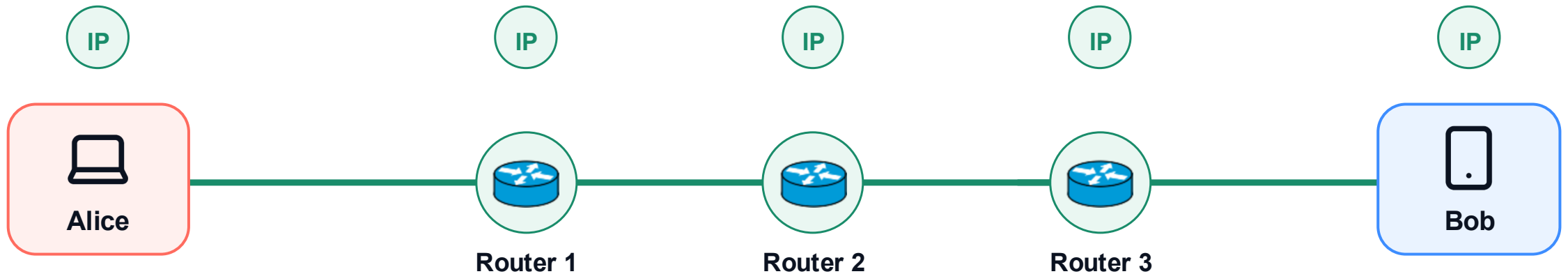
Transport peers are endpoints—not applications



The transport peer is the remote transport implementation serving that process.

Every IP node on the path processes IP

EVERY NODE ON THE PATH RUNS IP PROCESSING

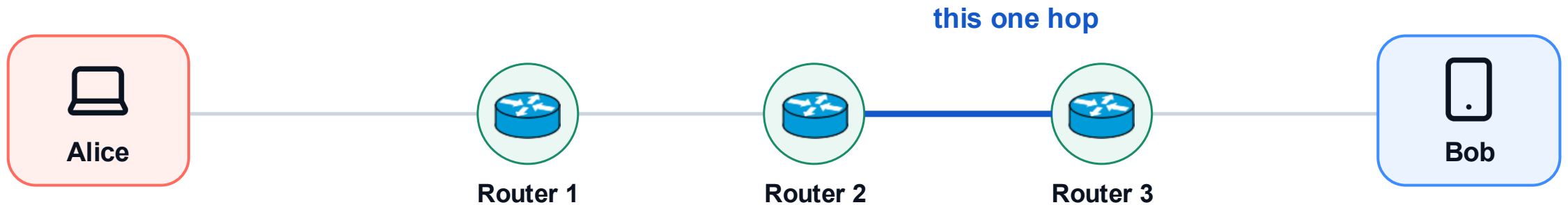


Every IP node processes the packet; routers are not all directly connected peers.

Link-layer peers are neighbors on one link

ONE LINK CREATES ONE NEIGHBOR RELATIONSHIP

Router 2 ↔ Router 3



A router has a different link-layer peer on each connected interface.